Movie Rec Hub: a Microservice Movie Recommendation Platform

PSIA Project Report - Group 34

Group members: 109358 - Gonçalo Torcato, 120202 - Sara Nascimento, 120232 - Sofia Rodrigues

Video: https://youtu.be/pLkDlg-nCRM

1 Introduction

Movie Rec Hub is a web application where users browse movies, write reviews, organise movies into lists and receive personalised recommendations. The recommendations are produced by a large language model (LLM) that takes into account each user's lists, reviews and favourite genres.

The application is built as a set of Spring Boot microservices, each with its own PostgreSQL database. It runs locally with Docker Compose and in the Google Cloud on a Kubernetes cluster created with Terraform and configured with Ansible. The system also includes an LLM-based recommendation service and a monitoring stack with Prometheus and Grafana. As an advanced feature, the application keeps its state across a full shutdown of the infrastructure.

This report presents the design and deployment of the base system, including monitoring (Section 2), the persistent state feature (Section 3) and an evaluation of throughput and latency as the number of clients grows (Section 4). The video linked above shows the system running in the cloud.

2 System Design and Architecture

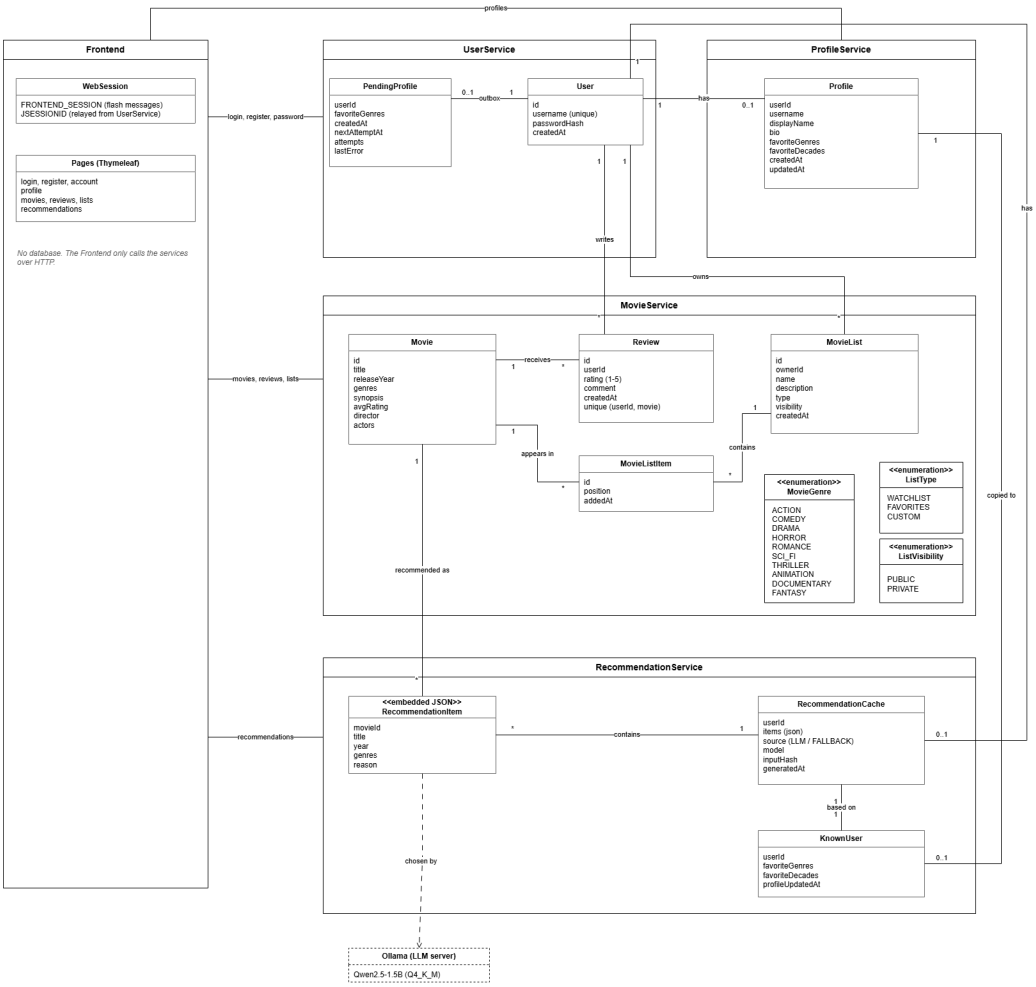


Fig. 1. Architecture of Movie Rec Hub.

The system is composed of five services that communicate through REST APIs:

- **Frontend:** the only entry point for users. It renders the web pages and calls the other services.
- **UserService:** accounts, login and sessions.
- **ProfileService:** user profiles, favourite genres and decades.
- **MovieService:** the movie catalogue, reviews and movie lists.
- **RecommendationService:** personalised recommendations using an LLM served by Ollama (Section 2.5).

Each service owns its database (users, profiles, movies and recommendations), so services never share tables and can be deployed and scaled independently.

2.1 Frontend

The Frontend is a Spring Boot application that renders pages on the server with Thymeleaf. A controller prepares the data in a Model and Thymeleaf fills an HTML template before it is sent to the browser, so the browser receives a complete page. Templates use `th:` attributes inside plain HTML: `th:if` shows a block only when a condition holds, `th:text` inserts values and `th:replace` includes reusable fragments, such as the sidebar, the alerts and the movie grid, defined once in a shared layout.

The Frontend talks to each backend service through a dedicated client class (for example, `MovieServiceClient`) with its own DTOs. The main pages are the movie catalogue with a genre filter, the movie details with reviews, the user's lists, login and registration, the profile pages and the "For you" recommendations page.

2.2 UserService and Testing

The UserService owns accounts and login sessions in its own PostgreSQL database. It exposes registration and login under `/api/auth` and password changes at `/api/users/me/password`. Usernames are trimmed and lower-cased, with uniqueness enforced by the database as well as the application. Passwords are stored as salted PBKDF2 hashes, never as plain text. Changing a password requires both an authenticated session and the current password.

Login creates a fresh session and returns a `JSESSIONID` cookie with `HttpOnly` and `SameSite=Strict`. The internal endpoint `GET /internal/auth/session` returns the authenticated user id, allowing ProfileService to verify the cookie without accessing the users database. Sessions are stored with Spring Session JDBC rather than only in application memory (Section 3).

Registration also saves a profile-creation job in `pending_profiles`, in the same transaction as the account. A background worker sends the username and initial favourite genres to ProfileService and retries failed deliveries. This lets users register even when ProfileService is temporarily unavailable. The job survives a restart, and repeated delivery does not overwrite an edited profile.

The service has 97 automated tests in 12 classes:

- **Unit tests (67):** JUnit 5 and Mockito cover request validation, JSON responses, the user model, password hashing, session handling and the HTTP client used for profile creation.
- **Integration tests (30):** `@DataJpaTest` and `@SpringBootTest` with real PostgreSQL cover database constraints and migrations, registration, login, password changes, account isolation, concurrent updates and the profile-creation queue, including failure, retry and rollback.

2.3 ProfileService and Testing

The ProfileService stores display names, biographies, favourite genres and favourite decades in its own PostgreSQL database. A profile uses the account id from UserService, without a foreign key across databases. Flyway migrations create and version the schema, while Hibernate validates it at startup.

Authenticated users read and partially update their own profile through `GET/PATCH /api/profiles/me`. The owner is selected from the verified session, rather than an id supplied in the request body. Omitted fields keep their previous values, and invalid fields or preferences are rejected. Users can also browse profiles with pagination, search by username and view another user's public profile under `/api/profiles`.

ProfileService forwards the session cookie to UserService for each protected request. An invalid session returns 401, while an unavailable UserService returns 503. Initial profiles are created through `PUT /internal/profiles/{userId}`. This operation is idempotent: retries leave existing biography and preferences unchanged. A background worker retrieves missing usernames for older profiles through UserService's internal API.

The service has 42 automated tests in 5 classes:

- **Session client (4 tests):** checks the cookie sent to UserService and distinguishes authentication failure from service unavailability using a local HTTP server.
- **Integration tests (38):** persistence and HTTP tests with PostgreSQL cover profile data, genres and decades, schema constraints, partial updates, ownership, public search and pagination, username backfill and concurrent or repeated initial creation. UserService calls are mocked in the HTTP endpoint tests.

Both services run unit tests with `mvn test` and integration tests with `mvn -Pintegration-tests verify`. A separate Python script creates two temporary PostgreSQL instances and runs both services to check registration during an outage, recovery after restart, repeated delivery and two workers processing the queue.

2.4 MovieService and Testing

The MovieService manages the movie catalogue, the reviews and the users' movie lists. It exposes a REST API under `/api/movies`, `/api/reviews` and `/api/lists` and stores its data in its own PostgreSQL database.

On first start, when the database is empty, the service fills the catalogue from The Movie Database (TMDB) API: it fetches the top-rated movies and stores, for each one, the title, release year, genres, synopsis, director and main actors.

The data model has four entities:

- **Movie:** title, release year, genres, synopsis, director, actors and the average rating.
- **Review:** a rating from 1 to 5 and a comment, written by a user about a movie. A user can review each movie only once (unique constraint on user and movie). Every time a review is created or deleted, the movie's average rating is recalculated.
- **MovieList:** a named list owned by a user, of type `WATCHLIST`, `FAVORITES` or `CUSTOM` and `PUBLIC` or `PRIVATE`. A user cannot have two lists with the same name.
- **MovieListItem:** a movie inside a list, with its position and the date it was added. The same movie cannot appear twice in one list.

The service enforces these rules itself. A private list is only visible to its owner and only the owner can change or delete a list. A global exception handler translates errors into clear HTTP codes. The API supports listing movies with a genre filter, searching by title, director or actor, filtering by year, creating and deleting reviews and creating, renaming, deleting and editing lists.

The MovieService follows a layered design: entities, repositories (Spring Data JPA), services with the business rules and REST controllers. It is covered by 156 automated tests in 10 test classes, each layer tested in isolation:

- **Controllers (53 tests):** `@WebMvcTest` with MockMvc, with the service layer replaced by mocks.
- **Services (53 tests):** JUnit 5 and Mockito, with the repositories mocked.
- **Repositories (39 tests):** `@DataJpaTest` on an in-memory H2 database, so no PostgreSQL instance is needed.
- **Entities (11 tests):** plain JUnit 5.

2.5 RecommendationService and Testing

The RecommendationService suggests movies in two ways: automatically, from what the user already likes and on demand, from a typed request such as "a drama movie". The choice is made by a small language model (Qwen2.5 1.5B) served by Ollama inside the same system, so no external AI service is used. It runs on CPU only, which limits its quality but keeps the deployment simple.

A background agent runs every 5 minutes and prepares 5 movies per user (the "For you" list), stored in the service's own PostgreSQL database; it skips users without signals and users whose taste has not changed (it compares a SHA-256 hash of the taste text). A typed request is sent to the model right away. Both are shown on the "For you" page of the Frontend.

The *taste* of a user is a short text built from the favourite genres and decades (ProfileService), the movie lists and the reviews (MovieService); movies already in lists or reviews are sent as "Already seen", so the model is asked not to repeat them. The prompt contains the whole catalogue (about 40 movies), the taste and the request and the model must answer in a fixed JSON format with movie ids. The service only checks the structure (ids exist, no duplicates, 1 to 5 movies).

The RecommendationService is covered by 66 automated tests in 10 test classes, each layer tested in isolation, without a database or Ollama:

- **Controller (8 tests):** `@WebMvcTest` with MockMvc, with the service layer replaced by mocks.
- **Services (33 tests):** JUnit 5 and Mockito, covering the recommendations, the background agent, the user's taste text, the known users and the catalogue refresh.
- **Engine (20 tests):** plain JUnit 5, covering the prompt builder, the validation of the model's answer and the taste hash.
- **Ollama client (5 tests):** Mockito, covering the request to the model and its errors.

Unit tests cannot judge the quality of a language model, so we also sent 22 typed requests to the real model (quantity, genre, decade, combined, typos and prompt injection) before and after improving the prompt. Invalid answers dropped from 2 to 0 and requests with the right number of movies rose from 14 to 20. The model handles quantity, combined requests and typos well, but fails with decades (0 of 4) and some single-genre requests. We attribute this to the small quantized model running on CPU, since further prompt changes brought no gain.

2.6 Deployment

Each service is packaged as a Docker image, built in two stages: Maven compiles the application in a build image and only the resulting JAR is copied to a smaller Java runtime image. Locally, Docker Compose starts the whole system with one command: the five services, their four PostgreSQL databases, Ollama, Prometheus and Grafana.

In Google Cloud, Terraform creates three virtual machines: one control plane and two workers, plus the persistent storage described in Section 3. The five application images are published to Google Artifact Registry. Ansible configures the machines, installs Kubernetes with kubeadm and Calico networking, and applies the manifests for the databases, Ollama and applications. The frontend is exposed on port `30510` of either worker; backend services and databases communicate through internal Kubernetes Services.

The deployment tools run in a local Docker management container. After configuring Google credentials and the TMDB key, publishing the images and preparing the cluster, `just deploy` installs the databases before the applications and checks their readiness. Monitoring is deployed separately (Section 2.7). The complete setup and update instructions are in `infra/README.md`.

2.7 Monitoring with Prometheus and Grafana

All five services expose metrics at `/actuator/prometheus` using Spring Boot Actuator and Micrometer, which required only two dependencies and three configuration properties. One of them enables a histogram of HTTP request durations, so that Grafana can compute latency percentiles (p50 and p95) instead of only the average. The average hides slow requests: if 95 requests take 50 ms and 5 take 2 s, the average is about 150 ms, a value no user actually experienced. The p50 shows the typical user's latency and the p95 shows the delay felt by the slowest 5% of requests, which is where performance problems appear first.

Prometheus collects the metrics every 15 seconds and Grafana displays them. Both are configured from files in the repository (provisioning), so the same dashboards appear automatically locally and in the cluster. We built two dashboards:

- **Services:** it shows availability, requests per second, latency (p50, p95 and maximum), the slowest endpoints, the error rate (4xx and 5xx), JVM memory, CPU, threads and database connection pools.
- **Cluster:** adapted from a community dashboard (`https://github.com/dotdc/grafana-dashboards-kubernetes`), with a section we added for the application: pods not ready, restarts per pod and free space on the database disk. It also shows CPU and memory per node and per namespace.

PRINTS

Fig. 2. Services dashboard in Grafana.

In Kubernetes, the monitoring stack runs in its own namespace and is deployed with an Ansible playbook that follows the structure of the application's deployment playbook. Because pods change IP address whenever they are recreated, Prometheus discovers its targets through the Kubernetes API instead of using fixed addresses. It collects metrics from four sources (Fig. 3): the Spring services, node-exporter (one pod per VM, for CPU, memory and disks), cAdvisor (built into each node's kubelet, for each container) and kube-state-metrics (the state of Kubernetes objects, such as ready pods and restarts).

DIAGRAMA AQUI

Fig. 3. Monitoring architecture in the Kubernetes cluster.

Security was considered throughout. Prometheus and kube-state-metrics have read-only permissions and kube-state-metrics can only read pods, nodes and namespaces. Prometheus has no login, so it is reachable only inside the cluster; external access goes through Grafana, which requires a password and blocks sign-up and anonymous access. The Grafana password is kept out of Git and passed to Kubernetes as a Secret, without being written anywhere outside `deployment.yml`.

3 Advanced Components: Persistent State

The advanced feature allows the entire Google Cloud cluster to be switched off and later restarted while keeping the application's saved data. Accounts, profiles and preferences, movie lists, reviews and automatic recommendations remain available after recovery. Users can continue using the application without recreating their accounts or entering their information again.

3.1 Database storage and sessions

The four PostgreSQL databases save their data on a dedicated persistent disk, separate from the operating system disk. Kubernetes connects each database to its own storage area on that disk. The data is therefore kept outside the application containers and can be reused when containers or machines restart. Terraform and Ansible automate the storage setup, preserving existing data on later runs.

UserService also stores login sessions in PostgreSQL. This allows a valid login to survive a service restart, as long as the browser still has the corresponding cookie. Sessions remain subject to the normal 30-minute inactivity timeout. A change to the application's public IP may also require the user to log in again, even though the account and its data have been preserved.

3.2 Stopping and recovering the cluster

Two commands automate the shutdown and recovery of the existing installation. Before acting, they check that the selected machines belong to the expected Google Cloud project and match the recorded installation.

- **Pause:** `just pause` stops the workers before the control plane, confirms that all three machines are off and keeps their disks and saved data.
- **Resume:** `just resume` starts the same machines, bringing the control plane back before the workers. It updates connection details when public addresses change and checks that storage, networking, databases, application services and Ollama have recovered. Monitoring services are also checked when installed.

The existing installation and saved data are reused, so recovery does not require a new deployment.

3.3 Validation and limitations

The cloud test confirmed that all three machines were switched off and later returned with the same identifiers. Existing accounts could log in again, and profiles, favourite genres and decades, watchlist contents and reviews were preserved. A comparison of the database records before and after shutdown also confirmed that the saved recommendations and their generation details were unchanged. This showed that previous results had been recovered, rather than generated again. A separate local test restarted UserService and PostgreSQL to verify that valid sessions could be recovered and that expired or replaced sessions were rejected.

The current design keeps recovery simple, but all four databases depend on worker-1 and its data disk. They cannot automatically move to another machine if that worker fails. Deleting the infrastructure with `terraform destroy` also deletes the data disk; recovering from disk loss would require a separate backup mechanism. The persistence guarantee covers the application data in PostgreSQL. Prometheus history, Grafana's internal data and downloaded Ollama models use temporary storage and are outside this guarantee; saved application recommendations remain in the database.

4 Evaluation

4.1 Method

To test the system we wrote a load generator in Java (`scripts/LoadTest.java`). It sends HTTP requests to the public address of the Frontend to simulate real users. Before the test starts the program creates one account per client and logs in with it, so that each client has its own session. Then, it runs several stages with an increasing number of clients (1, 2, 8, 32, 128 and 512). Each client is a thread that sends one request, waits for the answer and immediately sends the next one (no time in between respose-request). The pages are picked at random to cover all the services: the catalogue and movie details (MovieService), the user's lists, the profiles (ProfileService) and the recommendations page (RecommendationService) - a client also asks the model for recommendations (with a small probability), limited to 3 of these requests per client and per stage, because each one can take tens of seconds. Each stage has tens of seconds of warm-up (which we don't measure), followed by 60 seconds of measurement and a 20-second pause (so that we can tell the stages apart in Grafana). For each stage we report the throughput (successful requests per second), the median (p50) and 95th percentile (p95) of the latency, and the percentage of failed requests (any answer other than HTTP 200, or a timeout). Requests that are still waiting when the stage ends are not counted. We ran the test in both environment: in the cluster in Google Cloud and locally. During both runs we followed the system in the Grafana dashboard.

4.2 Results

```
39 filmes, 10 generos
18:51:24 - fase com 1 clientes
  5.7 pedidos/s, p50 128.6 ms, p95 232.8 ms, erros 0.00%
18:52:54 - fase com 2 clientes
  9.5 pedidos/s, p50 129.4 ms, p95 219.1 ms, erros 0.00%
18:54:24 - fase com 8 clientes
  8.7 pedidos/s, p50 137.8 ms, p95 417.7 ms, erros 0.00%
18:56:14 - fase com 32 clientes
  13.4 pedidos/s, p50 208.9 ms, p95 796.6 ms, erros 0.00%
18:59:26 - fase com 128 clientes
  47.2 pedidos/s, p50 1055.8 ms, p95 3313.9 ms, erros 0.00%
19:02:56 - fase com 512 clientes
  35.4 pedidos/s, p50 8261.2 ms, p95 11299.6 ms, erros 27.40%

clientes | pedidos | pedidos/s | p50 (ms) | p95 (ms) | erros (%)
       1 |     339 |       5.7 |    128.6 |    232.8 |     0.00
       2 |     570 |       9.5 |    129.4 |    219.1 |     0.00
       8 |     520 |       8.7 |    137.8 |    417.7 |     0.00
      32 |     805 |      13.4 |    208.9 |    796.6 |     0.00
     128 |    2830 |      47.2 |   1055.8 |   3313.9 |     0.00
     512 |    2927 |      35.4 |   8261.2 |  11299.6 |    27.40
```

(a) Cloud

```
PS C:\Users\sarad\git\g34> java scripts/LoadTest.java http://localhost:8080
39 filmes, 10 generos
11:32:43 - fase com 1 clientes
  47.3 pedidos/s, p50 15.8 ms, p95 30.3 ms, erros 0.00%
11:34:13 - fase com 2 clientes
  72.6 pedidos/s, p50 16.5 ms, p95 37.0 ms, erros 0.00%
11:35:43 - fase com 8 clientes
  4.2 pedidos/s, p50 17.7 ms, p95 17900.1 ms, erros 0.00%
11:37:38 - fase com 32 clientes
  12.3 pedidos/s, p50 20.2 ms, p95 43.0 ms, erros 0.00%
11:40:54 - fase com 128 clientes
  49.5 pedidos/s, p50 47.6 ms, p95 182.8 ms, erros 0.00%
11:44:22 - fase com 512 clientes
  62.9 pedidos/s, p50 500.2 ms, p95 801.8 ms, erros 7.39%

clientes | pedidos | pedidos/s | p50 (ms) | p95 (ms) | erros (%)
       1 |    2839 |      47.3 |     15.8 |     30.3 |     0.00
       2 |    4357 |      72.6 |     16.5 |     37.0 |     0.00
       8 |     252 |       4.2 |     17.7 |  17900.1 |     0.00
      32 |     735 |      12.3 |     20.2 |     43.0 |     0.00
     128 |    2969 |      49.5 |     47.6 |    182.8 |     0.00
     512 |    4072 |      62.9 |    500.2 |    801.8 |     7.39
```

(b) Local

Figure 1: Output of the load test: throughput (*pedidos/s*), p50 and p95 latency and error rate for each number of clients.

Latency. With few clients the latency in the cloud is about 130 ms, while locally it is about 16 ms (Figure 1). Most of this difference is the round trip over the Internet between the laptop and the cluster (in previous tests we observed that the Internet speed has a large impact on the results). Because each client waits for the answer before sending the next request, this also limits the throughput of a single client in the cloud to a few requests per second. Up to 32 clients the median latency stays almost constant in both setups, so the system is not yet under pressure. From 128 clients on, the latency in the cloud grows quickly (p50 of about 1 s and p95 of 3.3 s), and with 512 clients the median goes above 8 s.

Throughput and saturation. In the cloud, the throughput grows until 128 clients (47.2 req/s) and then drops with 512 clients (35.4 req/s), while 27.4% of the requests fail (most of them because they reach the timeout). This means that the system saturated somewhere between 128 and 512 clients (adding more clients was only making the queues longer). Locally, the error rate with 512 clients was lower (7.39%), as expected.